# Estructuras de Datos en Python

## 2/3 – Listas, tuplas y diccionarios (vs. arreglos y structs de C)

Informática y Programación

UNCuyo – Facultad de Ingeniería

9 de septiembre de 2026

# Índice de Contenidos


1. 1. Listas
2. 2. Tuplas
3. 3. Diccionarios
4. 4. Resumen y Conclusiones

# Dónde estamos

## Ya vimos (Presentación 1)

Sintaxis, tipos, condicionales, bucles, entrada de datos y funciones con parámetros simples.

## Lo que falta para resolver el TP4

Con variables sueltas alcanza para poco: los tres pisos del parking, el tablero de la batalla naval, el inventario de la librería, todos necesitan agrupar datos. En C eso se resuelve con `arreglos` y `structs`. En Python: **listas, tuplas y diccionarios**.

# Indice de Contenidos

1. 1. Listas
2. 2. Tuplas
3. 3. Diccionarios
4. 4. Resumen y Conclusiones

# Listas vs. arreglos de C

## Diferencia Principal: Tamaño

- **Un arreglo de C tiene tamaño fijo**, decidido al declararlo (o al reservar memoria con `malloc`).
- **Una lista de Python crece y se achica sola**: `append`, `insert`, `remove` cambian su tamaño en tiempo de ejecución, sin `malloc`/`realloc`.

**C**

```c
int notas[3] = {8, 5, 9};
int n = 3; // tamanio fijo
```

**Python**

```python
notas = [8, 5, 9]
notas.append(7)
# notas -> [8, 5, 9, 7]
```

# Indexado, `len()` y slicing

## `len()` en vez de llevar la cuenta a mano

En C solés guardar el tamaño en una variable aparte (`n`). En Python, `len(lista)` siempre está actualizado. El slicing (`lista[a:b]`) no tiene equivalente directo en C.

```
notas = [8, 5, 9, 7]

print(notas[0])      # primer elemento: 8
print(notas[-1])     # ultimo elemento: 7 (no existe en C)
print(len(notas))    # cantidad de elementos: 4
print(notas[1:3])    # sublista: [5, 9]
```

# Recorrer una lista

## Dos formas, según si necesitás el índice

Si no necesitás la posición, `for elemento in lista` es más directo que el `for` de C. Si necesitás el índice, `range(len(...))` lo replica.

### C

```c
int notas[3] = {8, 5, 9};
for (int i = 0; i < 3; i++) {
    printf("%d\n", notas[i]);
}
```

### Python

```python
notas = [8, 5, 9]
for nota in notas:
    print(nota)

# con indice, si hace falta:
for i in range(len(notas)):
    print(i, notas[i])
```

# Cuidado: aliasing con listas mutables

## El bug más común al armar matrices en Python

Multiplicar una lista con `*` no copia sus elementos si son a su vez listas: `[[0]*4]*4` crea **4 referencias a la misma fila**, no 4 filas independientes. Modificar una, modifica las cuatro. No tiene equivalente en C: `int t[4][4]` siempre reserva memoria contigua real, fila por fila.

# Aliasing: el ejemplo (tablero de la batalla naval)

```
# mal: las 4 filas son la MISMA lista, repetida por referencia
tablero = [[0] * 4] * 4
tablero[0][0] = 9
# tablero[1][0], [2][0] y [3][0] tambien cambiaron a 9!

# bien: una lista nueva por cada fila
tablero = [[0] * 4 for _ in range(4)]
tablero[0][0] = 9
# solo tablero[0][0] cambio
```

# Indice de Contenidos


1. 1. Listas
2. 2. Tuplas
3. 3. Diccionarios
4. 4. Resumen y Conclusiones

# Tuplas: listas inmutables

**Se parecen a una lista, pero no se pueden modificar**

Se escriben con paréntesis y, una vez creadas, no se les puede agregar, quitar ni reasignar elementos — como un arreglo de C declarado `const`, pero sin tamaño fijo previo.

```
punto = (3, 4)
print(punto[0])    # 3

punto[0] = 5       # TypeError: no se puede modificar una tupla
```

# Retorno múltiple: el verdadero motivo para usarlas

## En C, devolver más de un valor es incómodo

Una función en C solo devuelve **un** valor con `return`. Para devolver varios hay que usar punteros de salida (parámetros que la función modifica) o empaquetarlos en un `struct`.

### C (punteros de salida)

```c
void minmax(int v[], int n,
            int *min, int *max) {
    *min = *max = v[0];
    for (int i = 1; i < n; i++) {
        if (v[i] < *min) *min = v[i];
        if (v[i] > *max) *max = v[i];
    }
}
```

### Python (tupla)

```python
def minmax(v):
    return min(v), max(v)

# desempaquetado directo:
lo, hi = minmax([8, 5, 9, 7])
# lo -> 5, hi -> 9
```

# Indice de Contenidos


1. 1. Listas
2. 2. Tuplas
3. 3. Diccionarios
4. 4. Resumen y Conclusiones

# Diccionarios vs. structs de C

## Campos fijos vs. claves dinámicas

- Un `struct` de C define sus campos **en tiempo de compilación** — no se pueden agregar campos nuevos mientras corre el programa.
- Un `dict` de Python asocia claves con valores **en tiempo de ejecución** — se pueden agregar, quitar o consultar claves libremente.

### C

```c
typedef struct {
    int codigo;
    char nombre[50];
    float precio;
    int stock;
} Libro;

Libro l = {1, "Rayuela",
           15000.0, 10};
```

### Python

```python
libro = {
    "codigo": 1,
    "nombre": "Rayuela",
    "autor": "Cortazar",
    "precio": 15000.0,
    "stock": 10,
}
```

# Usar y recorrer un diccionario

## Acceso por clave, no por índice

Se lee y escribe con `d[çlave"]`, no con una posición numérica. Para recorrer todo el contenido, `.items()` da clave y valor juntos.

```
libro = {"codigo": 1, "nombre": "Rayuela", "stock": 10}

print(libro["nombre"])      # Rayuela
libro["stock"] -= 2         # vender 2 unidades

for clave, valor in libro.items():
    print(clave, "->", valor)
```

# Inventario completo: lista de diccionarios

## El patrón para el ejercicio 3 del TP4

Un `struct Libro[]` en C es una lista de diccionarios en Python: cada libro es un `dict`, y el inventario entero es una `list` de esos `dict`.

```
inventario = [
    {"codigo": 1, "nombre": "Rayuela", "stock": 10},
    {"codigo": 2, "nombre": "El Aleph", "stock": 3},
]

for libro in inventario:
    print(libro["codigo"], libro["nombre"], libro["stock"])
```

# Indice de Contenidos


1. 1. Listas
2. 2. Tuplas
3. 3. Diccionarios
4. 4. Resumen y Conclusiones

# Tabla Comparativa Rápida

| **Necesito...** | **Lenguaje C** | **Lenguaje Python** |
|---|---|---|
| Colección de tamaño fijo | `int v[N]` | lista (aunque no cambie de tamaño) |
| Colección que crece | `malloc/realloc` | lista con `append` |
| Colección inmutable | `const int v[N]` | tupla |
| Registro con campos fijos | `struct` | diccionario |
| Devolver varios valores | Punteros de salida o `struct` | tupla (`return a, b`) |

# Lo que sigue

## Con esto ya podés

Modelar los pisos del parking (listas), el tablero de la batalla naval (listas anidadas), y el inventario de la librería (lista de diccionarios) — la base de los ejercicios 5, 6 y 3 del TP4.

## Presentación 3: Cadenas, Archivos y Funciones Avanzadas

Falta lo que necesitan Wordle (comparar letra por letra), el Juego de la Vida (recorrer una grilla) y las estadísticas del archivo de texto (leer un `.txt` y contar palabras): **manejo avanzado de cadenas, lectura de archivos y funciones con retorno de colecciones**. Eso es lo que viene.

# ¡A programar en Python!